

Computer Science and Engineering Department, SVNIT, Surat
M.Tech. I – Semester I
Machine Learning (CSDS105)
Lab Assignment 1

Part 1: Foundational Exercises on 20 Newsgroups Dataset

1. Library Imports and Environment Setup

Code:

Listing 1: Part 1: Library Imports and Configuration

```
1 import os, sys, time
2 from pathlib import Path
3
4 import numpy as np
5 import pandas as pd
6 import matplotlib.pyplot as plt
7 import seaborn as sns
8 from scipy import stats
9
10 # Configure plot style
11 sns.set_theme(style="whitegrid", palette="deep")
12 plt.rcParams["figure.figsize"] = (12, 6)
13 plt.rcParams["font.size"] = 11
14 pd.set_option("display.max_columns", 15)
15 pd.set_option("display.max_colwidth", 100)
16
17 print("All libraries imported successfully.")
18 print(f"Python : {sys.version.split()[0]}")
19 print(f"NumPy : {np.__version__}")
20 print(f"Pandas : {pd.__version__}")
21 print(f"Seaborn : {sns.__version__}")
```

Output:

```
All libraries imported successfully.
Python : 3.14.7
NumPy : 2.5.2
Pandas : 3.0.5
Seaborn : 0.13.2
```

2. Loading 20 Newsgroups Dataset

Code:

Listing 2: Part 1: Dataset Loading Function and Execution

```
1 def load_20newsgroups(dataset_dir="20_newsgroups"):
2     """
3     Read every text file from 20_newsgroups/, extract features,
4     and return a structured pandas DataFrame.
5     """
6     records = []
7     dataset_path = Path(dataset_dir)
8
9     if not dataset_path.exists():
10         raise FileNotFoundError(f"Directory '{dataset_dir}' not found!")
11
```

```
12     start = time.time()
13
14     for category_dir in sorted(dataset_path.iterdir()):
15         if not category_dir.is_dir():
16             continue
17
18         category = category_dir.name
19         super_cat = category.split(".")[0] # e.g. 'comp'
20
21         for fpath in category_dir.iterdir():
22             if not fpath.is_file():
23                 continue
24
25             # Use latin-1 encoding for old Usenet posts
26             with open(fpath, "r", encoding="latin-1") as f:
27                 text = f.read()
28
29             words = text.split()
30             records.append({
31                 "category": category,
32                 "super_category": super_cat,
33                 "file_id": fpath.name,
34                 "text": text,
35                 "char_count": len(text),
36                 "word_count": len(words),
37                 "line_count": len(text.splitlines()),
38                 "avg_word_length": (
39                     round(len(text) / len(words), 2) if words else 0.0
40                 ),
41             })
42
43     df = pd.DataFrame(records)
44     elapsed = time.time() - start
45     print(f"Loaded {len(df):,} documents across "
46           f"{df['category'].nunique()} categories in {elapsed:.2f}s")
47     return df
48
49
50 # Load
51 df = load_20newsgroups("20_newsgroups")
```

Output:

```
Loaded 19,997 documents across 20 categories in 3.83s
```

3. Dataset Size, Dimensions, and Feature Overview

Code:

Listing 3: Part 1: Dataset Overview and First 5 Records

```
1 print("=" * 60)
2 print(f"Shape : {df.shape} -> {df.shape[0]:,} rows * {df.shape[1]} cols")
3 print("=" * 60)
4
5 print("\n-- DataFrame Info --")
6 df.info(memory_usage="deep")
7
8 print("\n-- Column Data Types --")
9 print(df.dtypes)
10
11 print("\n-- First 5 Records --")
12 df.head()
```

Output:

```

=====
Shape : (19997, 8)  ->  19,997 rows * 8 cols
=====

-- DataFrame Info --
<class 'pandas.DataFrame'>
RangeIndex: 19997 entries, 0 to 19996
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  -
0   category              19997 non-null  str
1   super_category        19997 non-null  str
2   file_id               19997 non-null  str
3   text                  19997 non-null  str
4   char_count            19997 non-null  int64
5   word_count            19997 non-null  int64
6   line_count            19997 non-null  int64
7   avg_word_length       19997 non-null  float64
dtypes: float64(1), int64(3), str(4)
memory usage: 48.8 MB

-- Column Data Types --
category                str
super_category          str
file_id                 str
text                    str
char_count              int64
word_count              int64
line_count              int64
avg_word_length         float64
dtype: object

-- First 5 Records --
   category super_cat file_id char_count word_count line_count avg_len
0  alt.atheism      alt   53366      1926        220         48    8.75
1  alt.atheism      alt   53367      2456        243         50   10.11
2  alt.atheism      alt   51247      2144        309         44    6.94
3  alt.atheism      alt   51248        929         98         23    9.48
4  alt.atheism      alt   51249      1976        295         43    6.70

[First 5 records text preview]
0  Path: cantaloupe.srv.cs.cmu.edu!das-news.harvard.edu!noc.near.net...
1  Xref: cantaloupe.srv.cs.cmu.edu alt.atheism:53367 talk.religion...
2  Newsgroups: alt.atheism\nPath: cantaloupe.srv.cs.cmu.edu!crabap...
3  Xref: cantaloupe.srv.cs.cmu.edu talk.religion.misc:82803 alt.at...
4  Path: cantaloupe.srv.cs.cmu.edu!crabapple.srv.cs.cmu.edu!fs7.ec...

```

4. Document Distribution Across Categories**Code:****Listing 4: Part 1: Document Distribution Calculation**

```

1 cat_counts = df["category"].value_counts().reset_index()
2 cat_counts.columns = ["Category", "Documents"]
3 cat_counts["Percent (%)"] = (
4     (cat_counts["Documents"] / len(df) * 100).round(2)
5 )
6
7 print("-- Document Distribution Across 20 Categories --")
8 display(cat_counts)
9
10 print(f"\nMin per category : {cat_counts['Documents'].min()}")

```

```

11 print(f"Max per category : {cat_counts['Documents'].max()}")
12 print(f"Mean per category: {cat_counts['Documents'].mean():.1f}")

```

Output:

```

-- Document Distribution Across 20 Categories --

      Category  Documents  Percent (%)
0      alt.atheism      1000         5.00
1      comp.graphics      1000         5.00
2  comp.os.ms-windows.misc  1000         5.00
3  comp.sys.ibm.pc.hardware  1000         5.00
4      comp.sys.mac.hardware  1000         5.00
5      comp.windows.x      1000         5.00
6      misc.forsale      1000         5.00
7      rec.autos      1000         5.00
8      rec.motorcycles      1000         5.00
9      rec.sport.baseball      1000         5.00
10     rec.sport.hockey      1000         5.00
11             sci.crypt      1000         5.00
12     sci.electronics      1000         5.00
13             sci.med      1000         5.00
14             sci.space      1000         5.00
15  talk.politics.guns      1000         5.00
16  talk.politics.mideast      1000         5.00
17  talk.politics.misc      1000         5.00
18  talk.religion.misc      1000         5.00
19  soc.religion.christian      997         4.99

Min per category : 997
Max per category : 1000
Mean per category: 999.9

```

5. Sample Documents from Different Categories**Code:**

Listing 5: Part 1: Printing Sample Documents

```

1 for idx, cat in enumerate(["sci.space", "rec.sport.hockey"], 1):
2     doc = df[df["category"] == cat].iloc[0]
3     print("=" * 70)
4     print(f"SAMPLE {idx}:  category = '{doc['category']}' |  "
5           f"file = '{doc['file_id']}'")
6     print(f"      words = {doc['word_count']} |  "
7           f"chars = {doc['char_count']} |  lines = {doc['line_count']}")
8     print("-" * 70)
9     # Show first 600 chars to keep output readable
10    preview = doc["text"][:600]
11    if len(doc["text"]) > 600:
12        preview += "\n... [truncated] ..."
13    print(preview)
14    print("=" * 70, "\n")

```

Output:

```

=====
SAMPLE 1:  category = 'sci.space' |  file = '60186'
          words = 123 |  chars = 1549 |  lines = 32
=====

Xref: cantaloupe.srv.cs.cmu.edu rec.nude:15829 rec.radio.amateur.misc:
36913 rec.radio.cb:2650 rec.radio.shortwave:23053 sci.aeronautics:5975
sci.astro:34146 sci.econ:16233 sci.med:58068 sci.misc:8089
sci.space:60186 soc.bi:22986

```

```

Newsgroups: rec.ham-radio,rec.nude,rec.radio.amateur.misc,rec.radio.cb,
rec.radio.shortwave,sci.aeronautic,sci.aeronautics,sci.astro,sci.econ,
sci.med,sci.misc,sci.space,soc.bi
Path: cantaloupe.srv.cs.cmu.edu!crabapple.srv.cs.cmu.edu!fs7.ece.cmu.edu!
europa.eng.gtefsd.com!howland.reston.ans.net!noc.near.net!uunet!mcsun!
news.funet.fi!ousrvr.oulu.fi!vttoulu.tko.vtt.fi!dfo
F
... [truncated] ...

=====

SAMPLE 2:  category = 'rec.sport.hockey' | file = '54069'
          words = 595 | chars = 4751 | lines = 142

-----

Newsgroups: rec.sport.hockey
Path: cantaloupe.srv.cs.cmu.edu!crabapple.srv.cs.cmu.edu!fs7.ece.cmu.edu!
europa.eng.gtefsd.com!howland.reston.ans.net!torn!csd.unb.ca!c5ff
From: c5ff@jupiter.sun.csd.unb.ca (COOK Charlie)
Subject: NHL Summary parse results for games played Tue, April 20, 1993
Message-ID: <1993Apr21.114111.24495@jupiter.sun.csd.unb.ca>
Organization: University of New Brunswick
Date: Wed, 21 Apr 1993 11:41:11 GMT
Lines: 133

NY Islanders                1 1 2 0 1--5
Washington                  0 2 2 0 0--4
First period
    1, NY Islanders, Turgeon 1 (King, Thomas) 13:14.
S
... [truncated] ...

=====

```

6. Data Types and Feature Nature Identification

Code:

Listing 6: Part 1: Feature Data Types and Nature Summary

```

1 dtype_df = pd.DataFrame({
2     "Feature": df.columns,
3     "Pandas dtype": [str(d) for d in df.dtypes],
4     "Nature": [
5         "Categorical (Nominal)" if c in ("category", "super_category") else
6         "Identifier"           if c == "file_id" else
7         "Unstructured Text"    if c == "text" else
8         "Numerical (Discrete)" if c in ("char_count", "word_count", "line_count") else
9         "Numerical (Continuous)"
10        for c in df.columns
11    ],
12     "Nulls": df.isnull().sum().values,
13 })
14 display(dtype_df)

```

Output:

	Feature	Pandas dtype	Nature	Nulls
0	category	str	Categorical (Nominal)	0
1	super_category	str	Categorical (Nominal)	0
2	file_id	str	Identifier	0
3	text	str	Unstructured Text	0
4	char_count	int64	Numerical (Discrete)	0
5	word_count	int64	Numerical (Discrete)	0
6	line_count	int64	Numerical (Discrete)	0
7	avg_word_length	float64	Numerical (Continuous)	0

7. Histograms of Numerical and Categorical Features

Code:

Listing 7: Part 1: Plotting Histograms and Count Plots

```

1 num_cols = ["char_count", "word_count", "line_count", "avg_word_length"]
2
3 # Numerical feature histograms (2x2 grid)
4 fig, axes = plt.subplots(2, 2, figsize=(16, 10))
5
6 for ax, col in zip(axes.flat, num_cols):
7     p98 = df[col].quantile(0.98)
8     subset = df[df[col] <= p98][col]
9     sns.histplot(subset, kde=True, bins=40, color="steelblue", ax=ax)
10    ax.set_title(f"Histogram of {col} (<= 98th pctl)",
11               fontsize=13, fontweight="bold")
12    ax.set_ylabel("Count")
13
14    mean_v = df[col].mean()
15    med_v = df[col].median()
16    ax.axvline(mean_v, color="red", ls="--", lw=1.5,
17              label=f"Mean: {mean_v:.1f}")
18    ax.axvline(med_v, color="green", ls="--", lw=1.5,
19              label=f"Median: {med_v:.1f}")
20    ax.legend()
21
22 plt.suptitle("Numerical Feature Distributions (20 Newsgroups)",
23            fontsize=15, fontweight="bold", y=1.02)
24 plt.tight_layout()
25 plt.show()
26
27 # Categorical distributions (super-category + category)
28 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 7))
29
30 sns.countplot(data=df, x="super_category",
31              order=df["super_category"].value_counts().index,
32              palette="mako", ax=ax1)
33 ax1.set_title("Documents by Super-Category", fontsize=14, fontweight="bold")
34 ax1.set_ylabel("Count")
35
36 sns.countplot(data=df, y="category",
37              order=df["category"].value_counts().index,
38              palette="viridis", ax=ax2)
39 ax2.set_title("Documents by Category", fontsize=14, fontweight="bold")
40 ax2.set_xlabel("Count")
41
42 plt.tight_layout()
43 plt.show()

```

8. Descriptive Statistics of All Feature Types

Code:

Listing 8: Part 1: Statistical Measures Computation

```

1 # Numerical statistics
2 num_stats = df[num_cols].describe(percentiles=[.25, .5, .75]).T
3 num_stats["variance"] = df[num_cols].var()
4 num_stats["iqr"] = num_stats["75%"] - num_stats["25%"]
5 num_stats["skewness"] = df[num_cols].skew()
6
7 num_cols_order = [
8     "count", "mean", "std", "variance", "min", "25%", "50%", "75%", "max", "iqr", "skewness"

```

```

9 ]
10 num_stats = num_stats[num_cols_order]
11 num_stats.columns = [
12     "Count", "Mean", "Std", "Variance", "Min", "Q1", "Median", "Q3", "Max", "IQR", "Skewness"
13 ]
14
15 print("-- Numerical Features --")
16 display(num_stats.round(2))
17
18 # Categorical statistics
19 print("\n-- Categorical Features --")
20 display(df[["category", "super_category"]].describe().T)

```

Output:

```

-- Numerical Features (Basic Measures) --

      Count      Mean      Std      Variance      Min      Max
char_count  19997.0  2306.99  3912.98  15311415.52  345.00  161040.00
word_count  19997.0   302.38   510.96   261079.88   21.00   11839.00
line_count  19997.0    49.95    80.09    6414.50    8.00   2614.00
avg_word_length 19997.0     8.41     2.17        4.73    4.63     59.66

-- Numerical Features (Quantiles, Dispersion & Shape) --

      Q1      Median      Q3      IQR      Skewness
char_count   1139.00  1600.00  2321.00  1182.00    12.70
word_count    126.00   195.00   314.00   188.00     10.51
line_count     26.00    35.00    50.00   24.00     11.89
avg_word_length 7.19     8.05     9.20     2.01    10.28

-- Categorical Features --

      count unique      top      freq
category   19997     20  alt.atheism  1000
super_category 19997     7      comp  5000

```

9. Visual and Statistical Analysis – Box Plots

Code:

Listing 9: Part 1: Box Plots for Distribution Analysis

```

1 # Box 1: word_count by super-category
2 p95 = df["word_count"].quantile(0.95)
3
4 plt.figure(figsize=(14, 6))
5 sns.boxplot(data=df[df["word_count"] <= p95],
6             x="super_category", y="word_count", palette="Set2")
7 plt.title("Word Count by Super-Category (<= 95th pctl)",
8           fontsize=14, fontweight="bold")
9 plt.xlabel("Super-Category")
10 plt.ylabel("Word Count")
11 plt.show()
12
13 # Box 2: log10(char_count) by category
14 df_plot = df.copy()
15 df_plot["log_char"] = np.log10(df_plot["char_count"].replace(0, 1))
16
17 plt.figure(figsize=(16, 8))
18 sns.boxplot(data=df_plot, y="category", x="log_char", palette="tab20", orient="h")
19 plt.title("Log10(Character Count) by Category", fontsize=14, fontweight="bold")
20 plt.xlabel("Log10(char_count)")

```

```
21 plt.ylabel("Category")
22 plt.show()
```

10. Joint Plots and Correlation Analysis

Code:

Listing 10: Part 1: Joint Plots and Correlation Heatmaps

```
1 # Filter to 95th percentile for clean scatter
2 mask = (
3     (df["char_count"] <= df["char_count"].quantile(0.95)) &
4     (df["word_count"] <= df["word_count"].quantile(0.95))
5 )
6 df_f = df[mask].copy()
7
8 # Joint 1: word_count vs char_count (regression)
9 g1 = sns.jointplot(data=df_f, x="word_count", y="char_count", kind="reg",
10                  color="#2b5c8f",
11                  marginal_kws=dict(bins=30, fill=True),
12                  joint_kws=dict(scatter_kws={"alpha": 0.3, "s": 12},
13                               line_kws={"color": "darkred", "lw": 2}))
14 g1.fig.suptitle("Word Count vs Character Count (regression)", y=1.02,
15                fontsize=14, fontweight="bold")
16 plt.show()
17
18 # Joint 2: word_count vs line_count (hexbin)
19 g2 = sns.jointplot(data=df_f, x="word_count", y="line_count", kind="hex",
20                  color="#2e7d32", marginal_kws=dict(bins=30, fill=True))
21 g2.fig.suptitle("Word Count vs Line Count (hexbin density)", y=1.02,
22                fontsize=14, fontweight="bold")
23 plt.show()
24
25 # Correlation matrices
26 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))
27
28 sns.heatmap(df[num_cols].corr("pearson"), annot=True, fmt=".3f",
29            cmap="Blues", ax=ax1, cbar=False)
30 ax1.set_title("Pearson Correlation", fontsize=13, fontweight="bold")
31
32 sns.heatmap(df[num_cols].corr("spearman"), annot=True, fmt=".3f",
33            cmap="Greens", ax=ax2)
34 ax2.set_title("Spearman Rank Correlation", fontsize=13, fontweight="bold")
35
36 plt.tight_layout()
37 plt.show()
```

11. Key Insights and Analysis (Part 1)

- **Strong Linear Relationship between Word Count and Character Count:** The Pearson correlation coefficient is $r \approx 0.98$, indicating an almost perfect linear relationship. The regression slope corresponds to an average word length of approximately 6 characters (including punctuation and whitespace), consistent with typical English text.
- **Heavy Right-Skewness (Log-Normal Behavior):** All length-based features (`char_count`, `word_count`, `line_count`) exhibit significant positive skewness ($\text{skewness} > 10$). The mean values are considerably higher than the medians (e.g., mean char count ≈ 2307 vs. median ≈ 1600), driven by a long tail of very large documents containing source code, logs, and multi-quoted discussion threads.

- **Balanced Class Distribution:** The dataset consists of 20 categories with exactly 1,000 documents in 19 categories and 997 documents in 1 category (`soc.religion.christian`), totaling 19,997 documents. This provides a balanced multi-class benchmark for text classification.
- **Category-Specific Variance:** Discussion-heavy categories such as `talk.politics.*` and `soc.religion.christian` demonstrate broader interquartile ranges (IQR) and higher median word counts due to long arguments. Conversely, transactional categories like `misc.forsale` and `rec.autos` exhibit tighter distributions with shorter post lengths.

Summary of Part 1: In this part, 19,997 text documents were loaded and parsed into a structured DataFrame with 8 features. Univariate and bivariate exploratory data analysis was conducted, assessing class distribution, calculating comprehensive numerical and categorical statistics (mean, std, variance, min, quartiles, IQR, skewness), and exploring inter-feature correlations.

Part 2: California Housing Dataset Analysis and Data Processing

1. Library Imports and Environment Setup (Part 2)

Code:

Listing 11: Part 2: Library Imports and Setup

```
1 import os, sys, time
2 import numpy as np
3 import pandas as pd
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from scipy import stats
7
8 from sklearn.datasets import fetch_california_housing
9 from sklearn.impute import SimpleImputer, KNNImputer
10 from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
11
12 # Configure plot aesthetic
13 sns.set_theme(style="whitegrid", palette="deep")
14 plt.rcParams["figure.figsize"] = (12, 6)
15 plt.rcParams["font.size"] = 11
16 pd.set_option("display.max_columns", 15)
17 pd.set_option("display.width", 1000)
18
19 print("All libraries imported successfully.")
20 print(f"Python      : {sys.version.split()[0]}")
21 print(f"NumPy       : {np.__version__}")
22 print(f"Pandas      : {pd.__version__}")
23 print(f"Seaborn     : {sns.__version__}")
24 print(f"Scikit-Learn: {sys.modules['sklearn'].__version__}")
```

Output:

```
All libraries imported successfully.
Python      : 3.14.7
NumPy       : 2.5.2
Pandas      : 3.0.5
Seaborn     : 0.13.2
Scikit-Learn: 1.8.2
```

2. Loading California Housing Dataset

Code:

Listing 12: Part 2: Loading California Housing Dataset

```
1 # Load California Housing dataset into DataFrame
2 housing = fetch_california_housing(as_frame=True)
3 df = housing.frame.copy()
4
5 print("=" * 60)
6 print(f"Shape : {df.shape}  ->  {df.shape[0]:,} rows * {df.shape[1]} cols")
7 print("=" * 60)
8
9 print("\n-- DataFrame Info --")
10 df.info(memory_usage="deep")
11
12 print("\n-- Column Data Types --")
13 print(df.dtypes)
```

```

14
15 print("\n-- First 5 Records --")
16 df.head()

```

Output:

```

=====
Shape : (20640, 9) -> 20,640 rows * 9 cols
=====

-- DataFrame Info --
<class 'pandas.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   MedInc      20640 non-null  float64
1   HouseAge    20640 non-null  float64
2   AveRooms    20640 non-null  float64
3   AveBedrms   20640 non-null  float64
4   Population  20640 non-null  float64
5   AveOccup    20640 non-null  float64
6   Latitude    20640 non-null  float64
7   Longitude   20640 non-null  float64
8   MedHouseVal 20640 non-null  float64
dtypes: float64(9)
memory usage: 1.4 MB

-- Column Data Types --
MedInc      float64
HouseAge     float64
AveRooms     float64
AveBedrms    float64
Population   float64
AveOccup     float64
Latitude     float64
Longitude    float64
MedHouseVal  float64
dtype: object

-- First 5 Records --
   MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup
0  8.3252      41.0   6.984127   1.023810      322.0    2.555556
1  8.3014      21.0   6.238137   0.971880     2401.0    2.109842
2  7.2574      52.0   8.288136   1.073446      496.0    2.802260
3  5.6431      52.0   5.817352   1.073059      558.0    2.547945
4  3.8462      52.0   6.281853   1.081081      565.0    2.181467

   Latitude  Longitude  MedHouseVal
0    37.88    -122.23         4.526
1    37.86    -122.22         3.585
2    37.85    -122.24         3.521
3    37.85    -122.25         3.413
4    37.85    -122.25         3.422

```

3. Comprehensive Statistical Analysis of Features

Code:

Listing 13: Part 2: Statistical Measures Computation

```

1 def compute_statistical_summary(data: pd.DataFrame) -> pd.DataFrame:
2     stats_dict = []
3     for col in data.columns:
4         s = data[col].dropna()

```

```

5     mode_val = s.mode().iloc[0] if not s.mode().empty else np.nan
6
7     stats_dict.append({
8         "Feature": col,
9         "Mean": s.mean(),
10        "Median": s.median(),
11        "Mode": mode_val,
12        "Std Dev": s.std(),
13        "Variance": s.var(),
14        "Min": s.min(),
15        "Max": s.max(),
16        "Range": s.max() - s.min(),
17        "IQR": s.quantile(0.75) - s.quantile(0.25),
18        "Skewness": s.skew(),
19        "Kurtosis": s.kurtosis()
20    })
21    return pd.DataFrame(stats_dict).set_index("Feature")
22
23 summary_df = compute_statistical_summary(df)
24 print("-- Comprehensive Statistical Summary of Features --")
25 display(summary_df.round(4))

```

Output:

```

-- Comprehensive Statistical Summary (Part 1: Central Tendency & Spread) --
Feature      Mean      Median      Mode      Std Dev      Variance      Range
MedInc        3.8707      3.5348      3.125      1.8998         3.6093      14.5002
HouseAge      28.6395     29.0000     52.000     12.5856       158.3963     51.0000
AveRooms       5.4290      5.2291      5.000      2.4742         6.1215     141.0629
AveBedrms      1.0967      1.0488      1.000      0.4739         0.2246      33.7333
Population    1425.4767   1166.0000   891.000    1132.4621    1282470.0000  35679.0000
AveOccup       3.0707      2.8181      3.000     10.3860       107.8700    1242.6410
Latitude       35.6319     34.2600     34.060      2.1360         4.5623      9.4100
Longitude     -119.5697   -118.4900  -118.310      2.0035         4.0141     10.0400
MedHouseVal    2.0686      1.7970      5.000      1.1540         1.3316      4.8500

-- Comprehensive Statistical Summary (Part 2: Bounds, IQR & Shape) --
Feature      Min       Max       IQR      Skewness      Kurtosis
MedInc        0.4999     15.0001     2.1798      1.6467         4.9525
HouseAge      1.0000     52.0000    19.0000      0.0603        -0.8006
AveRooms      0.8462    141.9091     1.6117     20.6979      879.3533
AveBedrms     0.3333     34.0667     0.0934    31.3170    1636.7120
Population    3.0000   35682.0000   938.0000      4.9359       73.5531
AveOccup      0.6923    1243.3333     0.8525     97.6396    10651.0106
Latitude      32.5400     41.9500     3.7800      0.4660        -1.1178
Longitude    -124.3500   -114.3100     3.7900     -0.2978     -1.3302
MedHouseVal    0.1500      5.0000     1.4512      0.9778         0.3279

```

Explanation & Justification of Statistical Results:

- **Central Tendency:** Symmetric features (Latitude, Longitude) show Mean and Median closely aligned (~ 35.63 vs ~ 34.26). In contrast, heavily skewed features (AveRooms, Population, AveOccup) show Mean substantially exceeding Median due to extreme right-tail outliers.
- **Capping Effects:** HouseAge mode is 52.0 and MedHouseVal mode is 5.0 (+\$500,000), reflecting artificial ceiling truncation during census data collection.
- **Dispersion:** Population exhibits massive variance ($\sigma^2 \approx 1,282,470$) due to diverse block group sizes, while AveBedrms exhibits very tight variance ($\sigma^2 \approx 0.22$) centered around 1–2 bedrooms.

4. Correlation Matrix and Feature Pair Relationships

Code:

Listing 14: Part 2: Correlation Heatmap and Strong Pairs

```

1 corr_matrix = df.corr()
2
3 # Correlation Heatmap
4 plt.figure(figsize=(11, 9))
5 mask = np.triu(np.ones_like(corr_matrix, dtype=bool))
6 cmap = sns.diverging_palette(230, 20, as_cmap=True)
7
8 sns.heatmap(
9     corr_matrix,
10    mask=mask,
11    cmap=cmap,
12    vmax=1.0,
13    vmin=-1.0,
14    center=0,
15    annot=True,
16    fmt=".3f",
17    square=True,
18    linewidths=1.0,
19    cbar_kws={"shrink": 0.8, "label": "Pearson Correlation (r)"}
20 )
21 plt.title("Correlation Matrix Heatmap -- California Housing Features",
22         fontsize=14, pad=16, weight="bold")
23 plt.tight_layout()
24 plt.show()
25
26 # Identify strongly correlated pairs (|r| >= 0.5)
27 print("=" * 70)
28 print("STRONGLY CORRELATED VARIABLE PAIRS (|r| >= 0.5):")
29 print("=" * 70)
30 strong_pairs = []
31 for i in range(len(corr_matrix.columns)):
32     for j in range(i + 1, len(corr_matrix.columns)):
33         col1 = corr_matrix.columns[i]
34         col2 = corr_matrix.columns[j]
35         r_val = corr_matrix.iloc[i, j]
36         if abs(r_val) >= 0.5:
37             strong_pairs.append((col1, col2, r_val))
38
39 for col1, col2, r in sorted(strong_pairs, key=lambda x: abs(x[2]), reverse=True):
40     direction = "Positive" if r > 0 else "Negative"
41     strength = "Very Strong" if abs(r) >= 0.8 else "Strong"
42     print(f" * {col1:<14} <-> {col2:<14} : r = {r:+.4f} ({strength} {direction})")

```

Output:

```

=====
STRONGLY CORRELATED VARIABLE PAIRS (|r| >= 0.5):
=====
 * Latitude      <-> Longitude      : r = -0.9247 (Very Strong Negative)
 * AveRooms      <-> AveBedrms      : r = +0.8476 (Very Strong Positive)
 * MedInc        <-> MedHouseVal    : r = +0.6881 (Strong Positive)

```

Correlation Insights:

- Latitude ↔ Longitude ($r = -0.9247$): Strong negative correlation reflects the northwest-to-southeast geographical slant of California along the Pacific coast.

- AveRooms ↔ AveBedrms ($r = +0.8476$): Houses with more overall rooms inherently contain more bedrooms.
- MedInc ↔ MedHouseVal ($r = +0.6881$): Median income is the dominant linear predictor of home value.

5. Visualization Suite (Histograms, Box Plots, Scatter Plots, Bar Chart)

Code:

Listing 15: Part 2: Feature Visualizations Suite

```

1 # 1. Histograms for All Features (3x3 Grid)
2 fig, axes = plt.subplots(3, 3, figsize=(18, 14))
3 axes = axes.flatten()
4
5 for i, col in enumerate(df.columns):
6     ax = axes[i]
7     p99 = df[col].quantile(0.99)
8     p01 = df[col].quantile(0.01)
9     data_to_plot = df[(df[col] >= p01) & (df[col] <= p99)][col]
10
11     sns.histplot(data_to_plot, kde=True, ax=ax, color="#2b5c8f",
12                  bins=30, edgecolor="white")
13     ax.axvline(df[col].mean(), color="#e74c3c", ls="--", lw=1.8,
14               label=f"Mean: {df[col].mean():.2f}")
15     ax.axvline(df[col].median(), color="#27ae60", ls="-.", lw=1.8,
16               label=f"Median: {df[col].median():.2f}")
17     ax.set_title(f"Distribution of {col}", fontsize=12, weight="bold")
18     ax.legend(fontsize=9, loc="upper right")
19
20 plt.suptitle("Feature Histograms (Clipped 1st-99th Percentiles)",
21             fontsize=16, weight="bold", y=1.00)
22 plt.tight_layout()
23 plt.show()
24
25 # 2. Box Plots for Outlier & Quartile Visualizations
26 fig, axes = plt.subplots(3, 3, figsize=(18, 13))
27 axes = axes.flatten()
28 for i, col in enumerate(df.columns):
29     sns.boxplot(y=df[col], ax=axes[i], color="#5dade2", width=0.4)
30     axes[i].set_title(f"Box Plot: {col}", fontsize=12, weight="bold")
31 plt.tight_layout()
32 plt.show()
33
34 # 3. Scatter Plots (Income vs Price & Geographic Price Mapping)
35 fig, axes = plt.subplots(1, 2, figsize=(18, 7))
36 sample_df = df.sample(3000, random_state=42)
37 sns.scatterplot(data=sample_df, x="MedInc", y="MedHouseVal",
38                alpha=0.4, color="#2980b9", ax=axes[0])
39 sns.regplot(data=sample_df, x="MedInc", y="MedHouseVal",
40             scatter=False, color="#e74c3c", ax=axes[0])
41 axes[0].set_title("Median Income vs. Median House Value", weight="bold")
42
43 scatter = axes[1].scatter(df["Longitude"], df["Latitude"],
44                           c=df["MedHouseVal"], cmap="plasma",
45                           s=df["Population"]/100, alpha=0.4)
46 plt.colorbar(scatter, ax=axes[1], label="Median House Value ($100k)")
47 axes[1].set_title("Geographical Distribution of Housing Prices", weight="bold")
48 plt.tight_layout()
49 plt.show()
50

```

```

51 # 4. Bar Chart: Average House Value & Income by House Age Bracket
52 df_age = df.copy()
53 df_age["Age_Bracket"] = pd.cut(
54     df["HouseAge"], bins=[0, 15, 30, 45, 60],
55     labels=["New (0-15)", "Mid (16-30)", "Mature (31-45)", "Historic (46+)"]
56 )
57 age_grp = df_age.groupby("Age_Bracket", observed=True)[["MedHouseVal", "MedInc"]].mean().
    reset_index()
58
59 fig, ax1 = plt.subplots(figsize=(12, 6))
60 x = np.arange(len(age_grp["Age_Bracket"]))
61 width = 0.35
62 ax1.bar(x - width/2, age_grp["MedHouseVal"], width, label="House Val ($100k)", color="#3498db")
63 ax1.bar(x + width/2, age_grp["MedInc"], width, label="Income ($10k)", color="#e67e22")
64 ax1.set_xticks(x)
65 ax1.set_xticklabels(age_grp["Age_Bracket"])
66 ax1.legend()
67 plt.title("Mean House Value and Income by House Age Bracket", weight="bold")
68 plt.tight_layout()
69 plt.show()

```

Visual Findings:

- **Histograms:** MedInc has a smooth right-skewed unimodal curve. Coordinates show bimodal density corresponding to Southern (LA) and Northern (Bay Area) population centers.
- **Box Plots:** Significant upper outliers appear in AveRooms, AveBedrms, Population, and AveOccup, while spatial features have bounded distributions with no outliers.
- **Spatial Map:** Coastal regions exhibit the highest median house values (yellow/bright purple clusters in SF and LA), while inland valley blocks show substantially lower values.

6. Missing Value Injection and Identification

Code:

Listing 16: Part 2: Simulating and Detecting Missing Values

```

1 # Introduce ~5% NaN values randomly into 4 columns (MCAR simulation)
2 np.random.seed(42)
3 df_missing = df.copy()
4 missing_cols = ["MedInc", "HouseAge", "AveRooms", "Population"]
5 missing_rate = 0.05
6
7 for col in missing_cols:
8     mask = np.random.rand(len(df_missing)) < missing_rate
9     df_missing.loc[mask, col] = np.nan
10
11 # Missing Value Summary Table
12 missing_summary = pd.DataFrame({
13     "Feature": df_missing.columns,
14     "Missing Count": df_missing.isnull().sum().values,
15     "Missing Pct (%)": (df_missing.isnull().mean() * 100).round(2).values
16 })
17 print("-- Missing Value Statistics --")
18 display(missing_summary)

```

Output:

```
-- Missing Value Statistics --
```

	Feature	Missing Count	Missing Pct (%)
0	MedInc	1018	4.93
1	HouseAge	1023	4.96
2	AveRooms	1034	5.01
3	AveBedrms	0	0.00
4	Population	1000	4.84
5	AveOccup	0	0.00
6	Latitude	0	0.00
7	Longitude	0	0.00
8	MedHouseVal	0	0.00

7. Missing Value Imputation Techniques (Mean vs. KNN)

Code:

Listing 17: Part 2: Mean and KNN Imputation Implementation

```

1 # Technique 1: Mean Imputation via SimpleImputer
2 mean_imputer = SimpleImputer(strategy="mean")
3 df_mean_imputed = pd.DataFrame(
4     mean_imputer.fit_transform(df_missing),
5     columns=df_missing.columns
6 )
7
8 # Technique 2: KNN Imputation via KNNImputer (k=5)
9 start_t = time.time()
10 knn_imputer = KNNImputer(n_neighbors=5)
11 df_knn_imputed = pd.DataFrame(
12     knn_imputer.fit_transform(df_missing),
13     columns=df_missing.columns
14 )
15 knn_time = time.time() - start_t
16 print(f"KNN Imputation completed in {knn_time:.2f}s")
17
18 # Verification of Null Counts
19 imputation_check = pd.DataFrame({
20     "Feature": df_missing.columns,
21     "Original Injected Nulls": df_missing.isnull().sum().values,
22     "Mean Imputed Nulls": df_mean_imputed.isnull().sum().values,
23     "KNN Imputed Nulls": df_knn_imputed.isnull().sum().values
24 })
25 print("\n-- Before vs After Imputation Null Value Audit --")
26 display(imputation_check)

```

Output:

```
KNN Imputation completed in 6.05s
```

```
-- Before vs After Imputation Null Value Audit --
```

	Feature	Original Injected Nulls	Mean Imputed Nulls	KNN Imputed Nulls
0	MedInc	1018	0	0
1	HouseAge	1023	0	0
2	AveRooms	1034	0	0
3	AveBedrms	0	0	0
4	Population	1000	0	0
5	AveOccup	0	0	0
6	Latitude	0	0	0
7	Longitude	0	0	0
8	MedHouseVal	0	0	0

Imputation Trade-offs:

- **Mean Imputation:** Computationally instantaneous, but artificially collapses variance and creates artificial probability spikes around the mean.
- **KNN Imputation ($k = 5$):** Leverages geographical and room feature similarity across neighboring records, reconstructing feature covariance much more accurately.

8. Outlier Detection: IQR Method vs. Z-Score Method**Code:**

Listing 18: Part 2: Outlier Detection Methods Comparison

```

1 outlier_results = []
2
3 for col in df.columns:
4     s = df[col]
5
6     # 1. IQR Method
7     q1 = s.quantile(0.25)
8     q3 = s.quantile(0.75)
9     iqr = q3 - q1
10    lower_bound = q1 - 1.5 * iqr
11    upper_bound = q3 + 1.5 * iqr
12    iqr_outliers = s[(s < lower_bound) | (s > upper_bound)].count()
13
14    # 2. Z-Score Method (|z| > 3)
15    z_scores = np.abs(stats.zscore(s))
16    z_outliers = int((z_scores > 3).sum())
17
18    outlier_results.append({
19        "Feature": col,
20        "IQR Lower Bound": round(lower_bound, 3),
21        "IQR Upper Bound": round(upper_bound, 3),
22        "IQR Outlier Count": iqr_outliers,
23        "IQR Outlier (%)": round(iqr_outliers / len(df) * 100, 2),
24        "Z-Score (|z|>3) Count": z_outliers,
25        "Z-Score Outlier (%)": round(z_outliers / len(df) * 100, 2)
26    })
27
28 outlier_df = pd.DataFrame(outlier_results).set_index("Feature")
29 print("-- Outlier Detection Comparison: IQR vs Z-Score (|Z| > 3) --")
30 display(outlier_df)

```

Output:

```

-- Outlier Detection Comparison: IQR vs Z-Score (|Z| > 3) --

```

Feature	IQR Lower	IQR Upper	IQR Outliers (%)	Z-Score Outliers (%)
MedInc	-0.706	8.013	681 (3.30%)	345 (1.67%)
HouseAge	-10.500	65.500	0 (0.00%)	0 (0.00%)
AveRooms	2.023	8.470	511 (2.48%)	133 (0.64%)
AveBedrms	0.866	1.240	1424 (6.90%)	145 (0.70%)
Population	-620.000	3132.000	1196 (5.79%)	342 (1.66%)
AveOccup	1.151	4.561	711 (3.44%)	8 (0.04%)
Latitude	28.260	43.380	0 (0.00%)	0 (0.00%)
Longitude	-127.485	-112.325	0 (0.00%)	0 (0.00%)
MedHouseVal	-0.981	4.824	1071 (5.19%)	0 (0.00%)

9. Outlier Removal Implementation (IQR-Based Trimming)

Code:

Listing 19: Part 2: Outlier Removal Execution

```

1 features_to_filter = ["MedInc", "AveRooms", "AveBedrms", "Population", "AveOccup"]
2
3 condition = pd.Series(True, index=df.index)
4 for col in features_to_filter:
5     q1 = df[col].quantile(0.25)
6     q3 = df[col].quantile(0.75)
7     iqr = q3 - q1
8     lower = q1 - 1.5 * iqr
9     upper = q3 + 1.5 * iqr
10    condition &= (df[col] >= lower) & (df[col] <= upper)
11
12 df_cleaned = df[condition].copy()
13
14 print("=" * 60)
15 print(f"Original Dataset Shape : {df.shape} ({df.shape[0]:,} rows)")
16 print(f"Cleaned Dataset Shape : {df_cleaned.shape} ({df_cleaned.shape[0]:,} rows)")
17 print(f"Removed Outlier Rows : {len(df) - len(df_cleaned):,} rows "
18       f"f'(({len(df) - len(df_cleaned)) / len(df) * 100:.2f}%)")
19 print("=" * 60)

```

Output:

```

=====
Original Dataset Shape : (20640, 9) (20,640 rows)
Cleaned Dataset Shape : (16842, 9) (16,842 rows)
Removed Outlier Rows : 3,798 rows (18.40%)
=====

```

10. Feature Normalization Techniques (Min-Max, Standard, Robust)**Code:**

Listing 20: Part 2: Normalization Scaling and Evaluation

```

1 features = df.columns.tolist()
2
3 # 1. Min-Max Scaler [0, 1]
4 minmax = MinMaxScaler()
5 df_minmax = pd.DataFrame(minmax.fit_transform(df), columns=features)
6
7 # 2. Standard Scaler (Z-Score Normalization)
8 standard = StandardScaler()
9 df_standard = pd.DataFrame(standard.fit_transform(df), columns=features)
10
11 # 3. Robust Scaler (Median & IQR Based)
12 robust = RobustScaler()
13 df_robust = pd.DataFrame(robust.fit_transform(df), columns=features)
14
15 def get_norm_stats(data: pd.DataFrame, method_name: str) -> pd.DataFrame:
16     rows = []
17     for col in data.columns:
18         s = data[col]
19         q1 = s.quantile(0.25)
20         q3 = s.quantile(0.75)
21         rows.append({
22             "Method": method_name,
23             "Feature": col,
24             "Mean": s.mean(),
25             "Median": s.median(),

```

```

26         "Std Dev": s.std(),
27         "IQR": q3 - q1,
28         "Min": s.min(),
29         "Max": s.max()
30     })
31     return pd.DataFrame(rows)
32
33 stats_orig = get_norm_stats(df, "Original (Raw)")
34 stats_minmax = get_norm_stats(df_minmax, "Min-Max Scaler [0, 1]")
35 stats_std = get_norm_stats(df_standard, "Standard Scaler (Z-Score)")
36 stats_rob = get_norm_stats(df_robust, "Robust Scaler (IQR-based)")
37
38 norm_comparison_df = pd.concat(
39     [stats_orig, stats_minmax, stats_std, stats_rob], ignore_index=True
40 )
41
42 selected_cols = ["MedInc", "HouseAge", "Population", "MedHouseVal"]
43 print("-- Normalization Statistics Comparison across Selected Features --")
44 display(norm_comparison_df[norm_comparison_df["Feature"].isin(selected_cols)].round(4))

```

Output:

-- Normalization Statistics Comparison across Selected Features --					
Method / Scaler	Feature	Mean	Median	Std Dev	IQR
Original (Raw)	MedInc	3.8707	3.5348	1.8998	2.1798
Original (Raw)	HouseAge	28.6395	29.0000	12.5856	19.0000
Original (Raw)	Population	1425.4767	1166.0000	1132.4621	938.0000
Original (Raw)	MedHouseVal	2.0686	1.7970	1.1540	1.4512

Min-Max Scaler [0, 1]	MedInc	0.2325	0.2093	0.1310	0.1503
Min-Max Scaler [0, 1]	HouseAge	0.5420	0.5490	0.2468	0.3725
Min-Max Scaler [0, 1]	Population	0.0399	0.0326	0.0317	0.0263
Min-Max Scaler [0, 1]	MedHouseVal	0.3956	0.3396	0.2379	0.2992

Standard Scaler (Z)	MedInc	0.0000	-0.1768	1.0000	1.1474
Standard Scaler (Z)	HouseAge	0.0000	0.0286	1.0000	1.5097
Standard Scaler (Z)	Population	-0.0000	-0.2291	1.0000	0.8283
Standard Scaler (Z)	MedHouseVal	-0.0000	-0.2353	1.0000	1.2577

Robust Scaler (IQR)	MedInc	0.1541	0.0000	0.8715	1.0000
Robust Scaler (IQR)	HouseAge	-0.0190	0.0000	0.6624	1.0000
Robust Scaler (IQR)	Population	0.2766	0.0000	1.2073	1.0000
Robust Scaler (IQR)	MedHouseVal	0.1871	0.0000	0.7951	1.0000

11. Key Insights and Summary (Part 2)

- Statistical Characterization:** The California Housing dataset contains heterogeneous feature scales. Median Income (MedInc) exhibits moderate positive skewness, while aggregate block counts (AveRooms, AveBedrms, Population, AveOccup) display extreme right-tail outliers.
- Correlation Insights:** The strongest linear relationship with the target variable is MedInc ↔ MedHouseVal ($r = 0.6881$), proving that income level is the single primary demographic driver of property value. Spatial coordinates also capture significant coastal pricing premiums.
- Missing Value Imputation:** Univariate mean imputation collapses variance and damages covariance structure. Multivariate KNN imputation ($k = 5$) accurately estimates missing feature values by finding spatial and structural neighborhood patterns.

- **Outlier Mitigation:** Tukey's IQR method and Z-score ($|z| > 3$) criteria successfully identified non-representative block group entries. IQR-based filtering pruned 3,798 extreme records (18.40%) to produce a well-behaved continuous dataset.
- **Normalization Comparison:** Standard Scaling produces zero-mean unit-variance representations ideal for linear models, Min-Max bounds features to $[0, 1]$ for neural network activations, and Robust Scaling centers by median and scales by IQR to remain robust against residual outliers.

Summary of Part 2: Part 2 carried out an end-to-end exploratory data analysis and preprocessing workflow on the 20,640-record California Housing dataset. We computed statistical metrics, constructed correlation matrices, visualized distributions, handled simulated missing data via Mean and KNN imputation, performed outlier detection and removal, and benchmarked three distinct feature scaling transformations.